



FIREBRAND

LYCEUM EDUCATION GROUP

Iteration and Looping Constructs

C# Pathway | Session 3

Session Recording & Professional Expectations

Session Recording

- All Estio training sessions are recorded to:
 - Support learner progress
 - Enable review of content
 - Maintain quality and compliance standards
- By attending this session, you acknowledge that recording is taking place.

Camera & Participation Expectations

- To support engagement and attendance monitoring:
 - Cameras should remain on during the session
 - Please participate actively in discussions and activities
 - If you are unable to use your camera or microphone for any reason, please inform your tutor directly via chat or message.

This session forms part of your professional apprenticeship experience. Please engage respectfully and professionally throughout.

Safeguarding, British Values & Wider Learning

Safeguarding protects individuals from harm, abuse, neglect, radicalisation and exploitation.

If you have a concern about yourself or someone else, email:

safeguarding@bpp.com

Or speak to your tutor privately.

We treat all concerns seriously and confidentially.

Safeguarding



The British Values are:

1. **Democracy**
2. **Rule of Law**
3. **Individual Liberty**
4. **Mutual Respect**
5. **Tolerance of Different Faiths and Beliefs**

These values underpin everything we do in professional and digital environments.

British Values



Prevent is about protecting people from being drawn into extremism or radicalisation.

In our programmes, Prevent links to:

- **Online safety**
- **Digital responsibility**
- **Critical thinking**
- **Professional ethics**

Prevent



As we go through today's session, consider:

How today's topic connects to British Values

Whether there are any Prevent or digital safety themes

How this learning supports your professional and career development

Your tutor will guide discussion and help you make these connections.

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

Lesson Aim and Objectives

Lesson Aim: Iteration and Looping Constructs

Lesson Objectives

Apply

- Apply looping constructs to automate repetitive tasks

Analyse

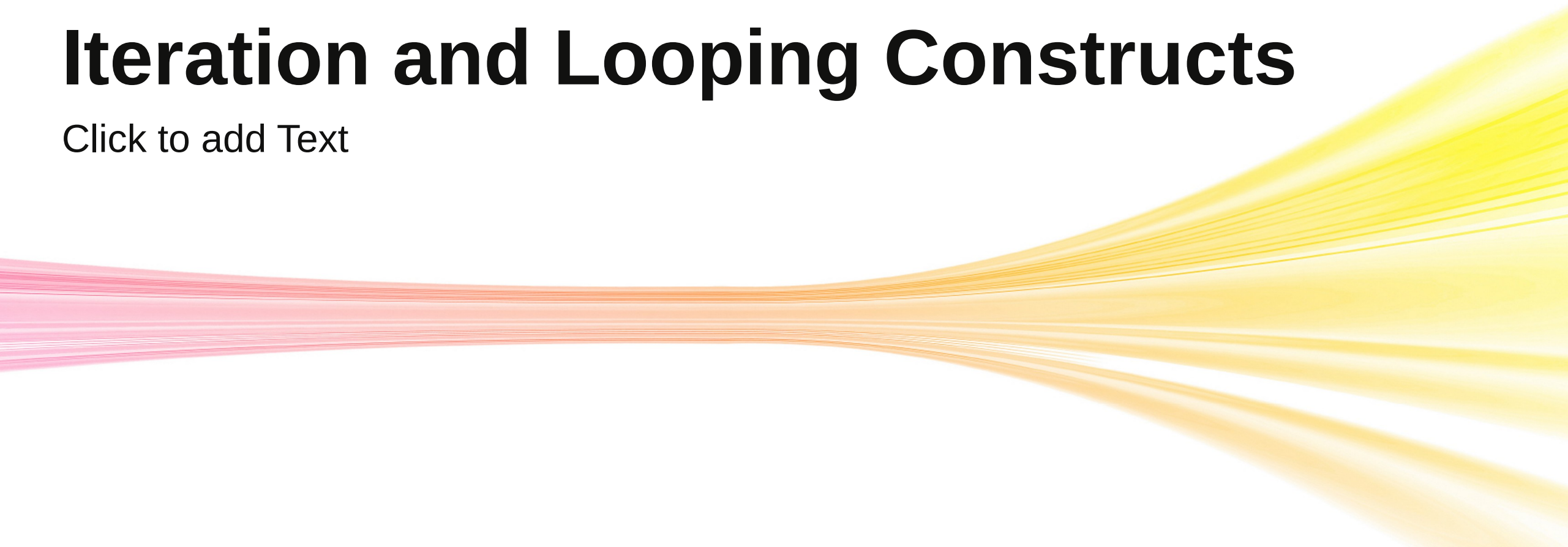
- Analyse loop execution and identify inefficiencies or errors

Evaluate

- Debug iteration issues including infinite loops and incorrect counters

Iteration and Looping Constructs

Click to add Text

A decorative graphic consisting of multiple thin, overlapping wavy lines that create a sense of motion. The lines are colored in a gradient from light pink on the left, through orange in the center, to bright yellow on the right. They originate from the bottom left and fan out towards the right side of the slide.

Introduction to Iteration

What is Iteration?

- **Repeated execution:** Running code blocks multiple times
- **Core block:** Essential foundation for algorithm design
- **Efficiency:** Eliminates redundant, copy-pasted code statements

The Two Loop Types

- **Definite iteration:** Loops run a predetermined number of times
- **Indefinite iteration:** Loops run until a condition changes

When to Use *For* vs. *While* vs. *Foreach*



The for Loop

- **Count-controlled:** Best for known iteration counts
- **Data structures:** Ideal for traversing arrays and lists
- **Safety:** Prevents accidental infinite execution loops

The while Loop

- **Condition-controlled:** Best for unknown duration tasks
- **True/False states:** Continues until a flag changes state
- **Flexibility:** Useful for user input validation steps

The foreach Loop

- **Collection-controlled:** Best for iterating through entire lists or arrays without managing counters
- **Data protection:** Implements a read-only variable state that prevents accidental structure changes
- **Safety:** Completely eliminates index boundary bugs and off-by-one runtime crashes

Loop Control Variables

Tracking Changes During Execution

State Tracking

- **Control variables:** Variables that dictate loop progression
- **Automatic updates:** for loops increment variables automatically
- **Manual updates:** while loops require explicit manual tracking

The Danger Zone

- **Infinite loops:** Occur when termination conditions are never met
- **Off-by-one errors:** Occur due to incorrect boundary comparison operators

The For Loop

Core Mechanics:

- **Explicit Counter:** Uses variable `i` to keep track of the current numeric index
- **Boundary Check:** Explicitly checks `i < employees.Length` before executing
- **Data Mutation:** Allows you to overwrite array items at specific index positions

```
string[ ] employees = { "Alice", "Bob", "Charlie" };
```

```
// Count-controlled loop using an index tracker
```

```
for (int i = 0; i < employees.Length; i++) {  
    Console.WriteLine(employees[i]);  
}
```

The While Loop

Core Mechanics:

- **External Control:** The tracking variables must be declared outside the loop structure
- **Manual Step Updates:** Requires an explicit calculation statement like `i++` inside
- **Input Validation:** Ideal for building interactive command menus with dynamic exits

```
string[] employees = { "Alice", "Bob", "Charlie" };
```

```
int i = 0; // 1. Initialise the counter variable manually
```

```
// Condition-controlled loop based on a Boolean rule
```

```
while (i < employees.Length) {
```

```
    Console.WriteLine(employees[i]);
```

```
    i++; } // 2. Critical update step to prevent infinite looping
```

The Foreach Loop

Core Mechanics:

- **Direct Access:** Pulls the actual data item directly out of the collection structure
- **No Index Counter:** Safely removes the need to track array length parameters manually
- **Read-Only Safety:** Protects data elements by blocking item adjustments during execution

```
string[ ] employees = { "Alice", "Bob", "Charlie" };
```

```
// Collection-controlled loop extracting items directly  
foreach (string name in employees) {  
    Console.WriteLine(name); }
```

Code Design: Good vs. Bad Loops

Poorly Designed Loop

// Inefficient nesting and unnecessary checks

```
for (int x = 0; x < 100; x++) {  
    for (int y = 0; y < 1; y++) {  
        if (x < 100) {  
            Console.WriteLine(x); } } }
```

- **Redundant steps:** Slows down total program execution speeds
- **Deep nesting:** Creates difficult, unreadable code blocks

Well-Structured Loop

// Clean, direct definite iteration

```
for (int x = 0; x < 100; x++) {  
    Console.WriteLine(x); }
```

- **Optimal performance:** Executes only necessary processor operations
- **High readability:** Clean structure simplifies peer review tracking.



FIREBRAND

LYCEUM EDUCATION GROUP

Guided Hands-On Exercises

Practice Task Checklist

Core Tasks

- **Count up:** Implement simple numeric counter sequences
- **List traversal:** Access elements sequentially inside arrays
- **Accumulate totals:** Calculate sums and running averages dynamically
- **Modify bounds:** Alter terminal conditions to observe changes

Expected Deliverables

- **Working scripts:**
Codes must execute without error flags
- **Aggregation logic:**
Correctly compute mathematical data totals

Lab Tasks: Putting Loops to Work

Exercise 1: The Countdown Challenge

- **Task:** Write a while loop that counts down from 5 to 1, then prints "Liftoff!"
- **Focus:** Practice manual control variable updates (count -= 1) to avoid infinite loops
- **Verification:** Ensure your loop stops executing after exactly 5 iterations

Lab Tasks: Putting Loops to Work

Exercise 2: Processing Data Lists

- **Task:** Create an array of 5 item prices. Use a for loop to look at each price and calculate a running total
- **Focus:** Accessing elements inside a data structure dynamically without hardcoding indexes
- **Verification:** Print the final total price at the very end of your script

Lab Tasks: Putting Loops to Work

Exercise 3: Aggregation & Averages

- **Task:** Expand Exercise 2 to calculate the **average** price of the items in the array
- **Focus:** Using the loop index or the .Length property alongside your accumulator total
- **Verification:** Test with numbers like [10, 20, 30] to see if it cleanly outputs an average of 20.0

Attendance Tracker System



ACTIVITY



Objective: Build a program that stores and summarises attendance data

Tasks / Requirements:

- Store attendee names in a list
- Use a loop to display all entries
- Count attendance and output totals

Key Point: Loop structures must correctly control iteration over the list

Extension Activity: Add validation to prevent empty or invalid entries



ACTIVITY



The Seeding Trap

The Common Logic Flaw (Anti-Pattern)

```
highestValue = 0;    // Breaks if all dataset items are negative!  
lowestValue = 0;    // BUG: 0 is smaller than any transaction!
```

The Glitch:

- If you seed lowestValue with 0, no positive transaction (like £5.50) will ever be smaller than it
- Your code will incorrectly report the lowest value as £0.00

The Robust Solution (Seeding from the Data)

// Always seed variables using actual values from your data array

```
highestValue = transactions[0];  
lowestValue = transactions[0];
```

Why it Works:

- By setting your initial baseline to the very first item in the list, you guarantee that your tracking variables stay grounded within the real boundaries of your dataset

The Race Condition

The Structural Concept:

- Modern enterprise systems execute code on multiple **Threads** operating at the same time to speed up processing routines.

The Failure State (Race Condition):

- Occurs when two or more distinct threads try to read and mutate a shared data variable simultaneously.
- The final value depends entirely on which thread completes its write operation last, leading to data corruption

Real-World Breakdown

Initial State: Balance = £100

Thread A reads balance (£100)

Thread A adds £50 (£150)

Thread A writes £150

→ Thread B reads balance (£100)

→ Thread B deducts £20 (£80)

→ Thread B writes £80 (Overwrites Thread A!)

Final Corrupted State: Balance = £80 (The £50 deposit is completely lost!)

Critical Sections & Locks

Critical Sections:

- The explicit block of code containing mutations to shared variables that must never be accessed by multiple threads at the same time

The Mutual Exclusion Lock (lock):

- Forces threads to queue up orderly. Only one thread can own the lock and enter the critical section at a time

C# Code Implementation

```
// Define a dedicated lock anchor object (System.Threading.Lock for .NET 9+)
private static readonly object _balanceLock = new object();
private static int _accountBalance = 100;

public static void DepositMoney(int amount) {
    // Intercepts competing threads and blocks simultaneous processing
    lock (_balanceLock) {
        // Critical Section: Only one thread can execute this math block at a time
        _accountBalance += amount;
    }
}
```

Best Practices

- **Don't Guess:**

Never use arbitrary baseline guess values like 99999 or -99999

- **Let the Data Set the Scale:**

Let the array itself establish the comparative baseline on line one

Data Processing Activity

Objective: Turning Raw Numbers into Actionable Business Insights

Tasks / Requirements:

- **Process arrays:** Loop through an e-commerce transaction dataset
- **Accumulate sums:** Track a running financial total
- **Compute averages:** Divide total revenue by order count
- **Extract extremes:** Identify highest and lowest individual values

Key Point: Use loops instead of built-in math shortcuts

Expected Outcome: Accurate data processing using loops with clear output demonstrating calculated insights



ACTIVITY



ACTIVITY



Next Activity will begin soon



Structured Debugging Lab



ACTIVITY



Common Issues Provided:

- Infinite loops
- Off-by-one errors
- Variables not updating correctly

Tasks:

- Trace execution step-by-step to identify the issue
- Annotate the root cause of the error
- Correct the script and re-run
- Classify the error type (logic or runtime)

Expected Outcome: Evidence of a structured debugging approach using tracing and reasoning



ACTIVITY





FIREBRAND

LYCEUM EDUCATION GROUP

Evaluation Discussion

Analysing Loop Design Choices

The Design Pillars

- **Readability:** How easily humans comprehend code layouts
- **Efficiency:** How optimally scripts utilise processing power
- **Maintainability:** How easily teams alter loops later

Task Requirements

- **Compare:** Contrast different iteration implementation styles
- **Justify:** Defend structural loop choices objectively
- **Connect:** Link layout directly to software performance

Measuring Code Quality vs. Performance

Readability Metrics

- Low cognitive load
- Minimal execution nesting
- Clear loop variable names

Efficiency Metrics

- Fewer total iterations
- No redundant inner calculations
- Optimal hardware memory usage

Team Maintainability Metrics

- Self-documenting logic steps
- Simple feature scalability paths
- Safe from regression errors

Explaining Loop Design Impact

Code Quality Insights:

- Clean code eliminates bugs
- Fewer statements lower failure rates
- Simple syntax survives team changes

Performance Realities:

- Poor loops waste processing time
- Redundant nesting scales very poorly
- Optimisation speeds up enterprise software

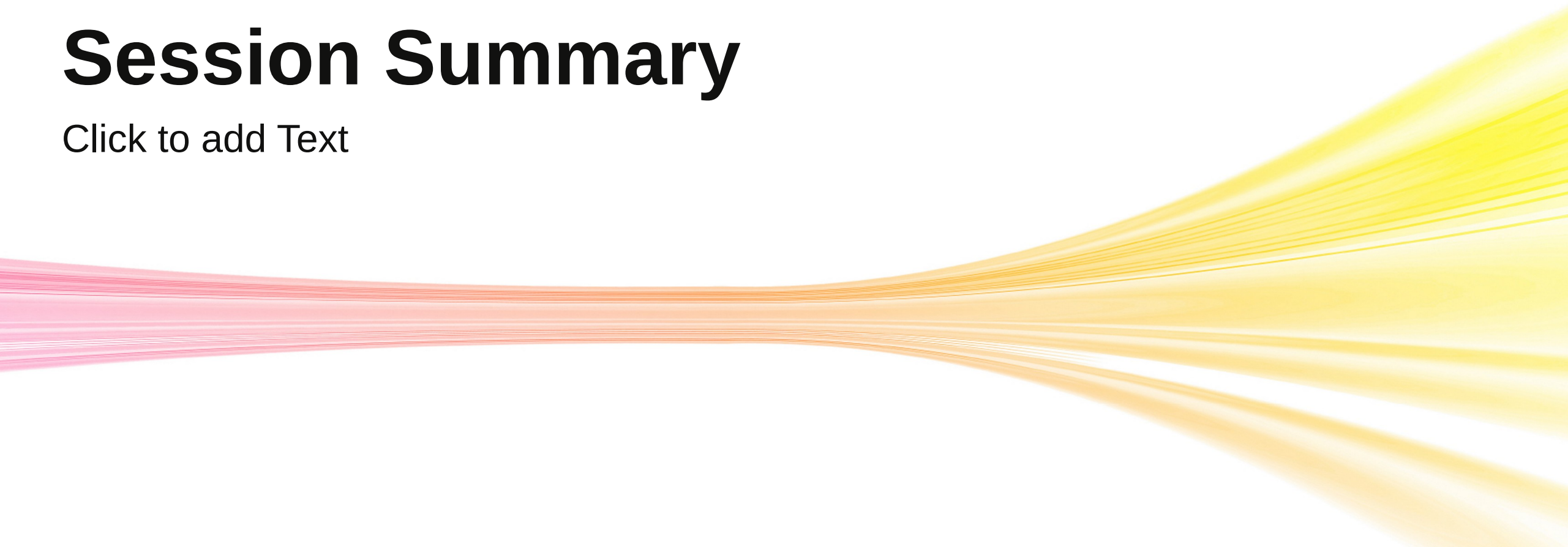
Stretch and Challenge

Improve your solutions by doing the following:

- Refactor inefficient loops into optimised versions
- Reduce unnecessary iterations
- Justify design improvements with reference to efficiency and readability

Session Summary

Click to add Text



Lesson Aim and Objectives

Lesson Aim: Iteration and Looping Constructs

Lesson Objectives

Apply

- Apply looping constructs to automate repetitive tasks

Analyse

- Analyse loop execution and identify inefficiencies or errors

Evaluate

- Debug iteration issues including infinite loops and incorrect counters

Competencies and Coverage

Professional Discussion underpinned by Portfolio

Knowledge

K1, K3, K4, K5, K7, K8,
K10, K12

Skills

S2, S3, S5, S8, S9, S13,
S14, S15, S17

Behaviours

B1, B4, B5, B6, B7, B8,
B9

Work-based Project with Questioning

Knowledge

K2, K6, **K9**, K11

Skills

S1, S4, S6, **S7**, S10,
S11, S12, **S16**

Behaviours

B2, B3

